GB Notion Frontend — Basket Reporting

The reporting form for the basket of a gas balloon. It keeps working without a signal, stamps every entry with the time and the position, carries the ballast figure forward across all devices, and lets several people report in parallel.

Seven kinds of entry — **Ballast, ATC, Ressources, Other**, and the three the app writes itself, **POS**, **MANPOS** and **PIC** — go into a JSON and a CSV file in a GitHub repository. Notion reads those files. Every device reads them back every five seconds, so each shows the complete flight.

When the satellites are jammed, a position can be given by eye on a map that works without a connection. When there is no link at all, everything is filed on the device and goes out by itself when the link returns.

One HTML file, no build step, no dependencies. Nothing leaves the device except calls to the GitHub API, the map tiles, and — if you leave place names on — one lookup per position.

Contents

1. The screen
2. Set up
3. Install on the iPad
4. Floating over other apps
5. Who is reporting
6. Ballast
7. ATC, Ressources, Other
8. Position
9. Automatic position reports
10. The Ops Normal clock
11. Place names
12. When the satellites are gone
13. WhatsApp
14. Master and followers
15. Several devices at once
16. What carries over
17. Sending, and what happens without a link
18. Working without a link, and GitHub's limits
19. The log
20. What the app writes
21. Polling from Notion
22. Beginning a new flight
23. Language
24. Keeping the crew on the current build
25. When a change does not appear
26. Limits worth knowing before takeoff
27. Version
28. License
29. Files

1. The screen

The whole app fits one phone screen without scrolling, and its outline is always visible. Only the log and the setup scroll.

The day/night switch is a square button at the top right of the menu, level with the word *Menu*, and nowhere else: choosing the colour is a thing one does between flights, and a button that turns the whole display red has no business under a thumb reaching for the post button, and shows the mode it leads to: a moon by day, a sun at night. **Every start is a day start** — night is chosen for a night, and a fresh launch is far more often a new day than the continuation of one.

Day mode is dark anthracite on a grey ground; unselected buttons carry a visible outline and a white face. Night mode is red on black and preserves dark adaptation. The sun/moon button in the header switches in one tap and the choice is remembered; on first launch the app follows the iPad's own appearance setting.

The header shows UTC to the second with the ballast still on board directly beneath it, right aligned with the seconds and in the same size, then the flight and callsign with the live navigation line beneath them — `293° · 12.3 kn · 3937 ft · +1.4 m/s` — course over ground, ground speed, altitude and rate of climb, in the order a pilot reads them. The accuracy of the fix has moved to the footer, where it sits beside a red, amber or green dot: green under 15 metres, amber under 50, red beyond that or with no fix at all. The rate of climb is figured from the altitude of two fixes at least five seconds apart and smoothed, because the raw difference between two consumer fixes jumps far too much to read. Before a fix it reads `GPS searching`; with a fix but no movement the course and speed fall away and only the accuracy remains. Then two buttons: menu and minimise. Below it, flush against the header, sit the four message types; the kilograms on board appear under them only on the Ballast tab, where they are relevant. There is no logo and no title inside the app — the home screen icon already says what this is, and the room is better spent on the form.

Everything that is not reporting lives behind the menu, and the menu is a list of **places** rather than a list of actions: *Return to Reporting Screen …* with a back arrow at the top, *Open log*, *Settings*, *Log — reload, print, export*, *Share setup with another device* on the master, and *About — more info*, which opens this manual in a window of its own with the licence linked from its last section.

Everything that concerns the log sits behind one entry, *Log — open, reload, print, export, clear*, so the main list stays short: opening it, reloading it from GitHub, printing it, exporting it and emptying it.

Clear the log is there for the test messages exchanged before a flight: the crew tries the whole chain, sees the rows arrive in Notion, and then wipes them so the flight starts on a clean sheet. It asks for the **master password** and then twice for confirmation. The table is read in full first, so a row written on another device and not yet pulled cannot survive the wipe, and every row is marked as deleted rather than merely removed — that is what stops it coming back from another device on the next sync. The flight itself, the setup and the archived flights are untouched. A device that is offline at that moment keeps whatever it has not yet sent; bring it online before wiping. The day/night switch has become a square button at the top right of the menu, level with the word *Menu*: it is a setting, not a destination, and it no longer takes a line of its own.

Send now has gone. Entries go out by themselves the moment they are made and again as soon as a lost connection returns; when something really is stuck, *Synchronize data* at the top of the setup does the job properly — it clears the cached ETags, pushes, reads the whole table back and then says what happened. A second button that only said *try again* was one button too many. The switch is named for the mode it takes you to, so it reads *Night mode* by day and *Day mode* at night.

Keeping the header to two buttons leaves the whole width for the flight and the navigation line, and the three screens are always one tap apart through the menu.

Night mode comes in four colours — red, amber, green or dimmed white — chosen in the setup. Red preserves dark adaptation best; the others are there for personal preference and for screens where red reads badly.

The app icon and the favicon are the tally itself — four navy strokes struck through in red, drawn slightly out of true so it reads as counted by hand rather than printed. The favicons have a transparent ground so they sit in a browser tab of any colour; the home screen icons stay opaque, since iOS renders transparency in an app icon as black. The set covers all three platforms: `apple-touch-icon` in 120, 152, 167 and 180 pixels for iPhone and iPad, manifest icons in 192 to 512 for Android and Chrome, two maskable versions with the artwork inside the safe circle so Android's adaptive shapes do not clip it, a `mask-icon.svg` for pinned tabs in desktop Safari, and a multi-resolution `favicon.ico`. The icons are opaque — iOS renders transparency in a home screen icon as black — and the manifest background is white to match, so the Android splash shows no seam. The 16 and 32 pixel favicons carry a simplified version with three uprights instead of four; at that size the narrower gaps of the full mark close up into a smudge.

2. Set up

The GitHub side — the data repository, the token and the Notion polling — is a separate document: **Setup guide**. What follows is the short version.

Hosting

1. Put these files in the repository root.
2. Settings -> Pages -> Source: `Deploy from a branch`, branch `main`, folder `/ (root)`.
3. The app is served at `https://<owner>.github.io/<repo>/` after about a minute.

Token

Settings -> Developer settings -> Personal access tokens -> Fine-grained tokens:

– Repository access: the data repository only
– Permissions -> Repository permissions -> **Contents: Read and write**
– Short expiry, for example one week after the flight

In the app

The *VFR squawk* field is the conspicuity code the VFR button writes; `7000` unless your airspace uses something else.

The *Ballast on board at start* field is only needed before the first inventory: it seeds the running figure so that early drops count, and it gives the first inventory something to compare against in `tally_diff_kg`. Leave it empty and the ballast figure simply starts with the first Take Inventory.

The reporter name for this device sits at the top of the setup and needs no password — it changes often enough, and getting it wrong only mislabels entries. Everything below it is locked: press *Unlock settings* and enter **1234**. Fill in the device mode and name, the flight, the two pilots, the weight of one ballast bag, the full ready-ballast weight, and the quick drop amounts. The methanol level at the start of the flight is a setting of its own — a tank is not always full when the balloon leaves — and it is what the methanol button shows until the first Ressources entry says otherwise.

An empty cylinder table on saving means the rows were never filled in, not that the bottles are gone: the previous entry is kept, the same rule that protects the GitHub token.

The oxygen cylinders are a table of up to four, each with its volume and its pressure when full. Every input has its name above it and its unit inside it.

Opening the **WhatsApp message layout** or the **GitHub connection** asks first — *Do you really want to change these settings?* Those two are where a slip costs a flight rather than a keystroke.

The GitHub connection and the WhatsApp recipients sit in collapsed sections at the bottom — open them once per device, then close them again. Saving asks twice before it applies, so a flight in progress cannot be overwritten by a stray tap. Leaving the setup screen locks it again.

3. Install on the iPad

Safari -> Share -> **Add to Home Screen**. It is saved as *Basket Reporting* and launches full screen without Safari's bars, and location and wake lock work as expected. Location permission is requested on the first entry; "While Using the App" is enough. Without it, entries are still recorded, with `gps_fix` set to `false`.

4. Floating over other apps

The minimise button collapses the app to a blue pill reading `HB-QWV Reporting` with the ballast still on board beneath it. Where that pill can float depends entirely on the platform, and it is worth being exact about it.

No web app can draw over other apps on iPadOS, iPhone or Android. That capability is reserved for native apps — on Android it needs the `SYSTEM_ALERT_WINDOW` permission, on iOS it does not exist at all. A browser page, installed or not, cannot paint outside its own window. Anything claiming otherwise on a web page is drawing inside that page.

So the pill floats inside the app window, and the app window is what the platform puts on top:

Platform	How to keep it above the other app
iPad	Install to the Home Screen, then drag it from the Dock onto the running app — **Slide Over** gives a narrow window floating over a map or Foreflight, swiped off the edge and back with one gesture. **Split View** and **Stage Manager** are the fixed and the free-floating variants.
Android, Samsung	Install to the Home Screen, then open it in **Pop-up view** from the Edge panel or the recents menu. An installed web app counts as an app there and gets a real floating window.
Android, others	**Split screen** from the recents menu. Free-floating windows exist only on Samsung and on devices in desktop mode.
Chrome on a laptop	The minimise button opens a genuine always-on-top window through document picture-in-picture — the only browser feature that does this. It survives switching to any other application. Useful for the chase car.

The app detects the picture-in-picture case on its own: press minimise and you get the always-on-top window where the browser supports it, and the in-app pill everywhere else. Clicking either brings the app back.

5. Who is reporting

The setup asks whether this is a **personal device** or a **shared device**.

On a personal device — the default, and what the two pilots' own phones should be set to — every entry is filed under the name entered there, whoever happens to be pilot in command. The reporter button is hidden; there is nothing to get wrong.

Switching between personal and shared takes effect at once: the reporter button appears beside *Report to CC* on all four forms, or disappears again, without leaving the setup.

On a shared device the reporter defaults to whoever is pilot in command, and a button beside *Report to CC* — about a third of the width — **walks through everyone who might be filing**: both pilots, and the name this device carries if it belongs to neither. A crew member on board who is not one of the two named pilots therefore needs no setting of their own; enter the name once as the reporter of the device and the button offers all three. It reads *reporting as* with the name below, and appends *(PIC)* when the two are the same person, so a glance tells you both facts at once. It turns dark while the alternate is active, so the exception is visible, and every entry carries that name until it is switched back. Changing the PIC on the Ressources tab moves the default with it.

Each row also records `device_mode`, so it is possible to tell afterwards which reporting regime a row came from.

6. Ballast

The kilogram field starts empty every time the Ballast tab is opened. A running total that carried over from ten minutes ago would be added to by mistake far more often than it would save a keystroke.

Ballast is shown in **whole kilograms everywhere** — in the header, the log, the messages and the printout. Ballast is counted in sacks, and a figure like `392.3 kg` would suggest a precision the basket does not have. The Schütte is rounded **down**, so its share is never overstated. The stored values keep whatever was entered; only the display is rounded.

Sand and water are carried forward separately. An inventory sets both, a drop reduces the one it was made of, and every entry therefore knows what is left of each — recorded as `sand_left_kg` and `water_left_kg` beside the total. An inventory reads `sand 390 kg („Schütte" 100%) · water 20 kg · 24 bags` — the Schütte named separately as a percentage and counted inside the sand figure, since that is where it physically is. A drop reads `dropped 40 kg sand · remaining 370.0 kg total ballast (350.0 kg sand, 20.0 kg water)`, and a water drop is stated in litres, `dropped 20 l water`, because that is how it is measured in the basket.

Drop subtracts what was thrown and records whether it was sand or water; sand is the default. The quick amounts **add up**: tapping 10 three times gives 30, which is how ballast actually leaves a basket — sack by sack, counted as you go. A *Reset* button appears beside the total the moment there is something to undo. The `kg` at the right edge of the button row names the unit once, so the buttons themselves stay bare numbers. The quick buttons only fill the field; posting is always the one dark button, so a knock against the iPad cannot log a drop.

Take Inventory records what is actually on board and resets the running figure. Three inputs:

- *Bags* and *Water* on one line — bags counted in total including safety ballast and multiplied by the weight per bag from the setup, water in litres at one kilogram per litre.
- *Ready ballast* („Schütte") — the steps come from the setup, `0,10,25,50,75,100` by default, as a percentage of the full ready-ballast weight, normally 30 kg. The last value used stays selected.

The running total under the fields shows the result before it is posted, and the entry lands in the table as three columns:

Column	Contents
`sand_kg`	bags x weight per bag, **plus** the ready ballast
`water_kg`	litres
`total_ballast_kg`	sand + water, and the new figure on board

Ready ballast is always sand, so sand + water = total. `inv_bags` and `inv_ready_pct` are carried alongside so an inventory can be reopened and corrected later.

`tally_diff_kg` **records where the figures parted company.** Every inventory stores the counted total minus what the running calculation expected at that moment. A reading of -60 means sixty kilograms left the basket without being logged; a positive figure means a drop was logged that did not happen, or a bag was miscounted. The first inventory has no predecessor to compare against and leaves the column empty unless a start ballast was entered in the setup. The figure is recalculated over the whole flight after every edit, deletion or merge, so it always reflects the current state of the table. The app also shows the expected value and the difference live, before an inventory is posted.

The dropped figure is measured from the first inventory, not from the sum of the drops: `first inventory total - currently on board`. Throwing sand is not exact; counting sacks is. Every later inventory therefore feeds straight into the dropped figure, including whatever went overboard without being logged.

7. ATC, Ressources, Other

ATC puts the station on its own line, then FREQ, SQ and a VFR button side by side, then the message.

A frequency must lie between **118.000 and 136.975 MHz** and be a real channel name. Both spacings are accepted: the 25 kHz channels ending in `00`, `25`, `50`, `75`, and the 8.33 kHz channels inserted between them, ending in `05 10 15`, `30 35 40`, `55 60 65`, `80 85 90`. The four endings `20`, `45`, `70` and `95` exist in neither scheme — `118.020` is neither a frequency nor a channel number — so they are refused as what they almost always are, a mistyped digit.

The **frequency** places its own point: after the third digit one appears as you type, so `12345` reads `123.45` on screen while you are still typing and settles to `123.450` when you leave the field. `1187` becomes `118.700`, `118` becomes `118.000`. A point or comma you type yourself is ignored — the field only ever counts digits — so both habits work.

The **squawk** is four octal digits — 8 and 9 simply do not appear as you type, and a code of one to three digits is refused on posting. The emergency codes are listed in the setup, `7500`, `7600` and `7700` by default; entering one brings up a confirmation naming what it means before the entry goes out, because they are three keystrokes away from ordinary codes.

The **VFR** button fills the squawk with the conspicuity code from the setup, `7000` by default, and lights up while that code is set. Tapping it again clears the field; typing anything else turns the light off. The station is typed as it reads — `Bern Info`, not `BERN INFO`. The keyboard capitalises the first letter of each word and the app no longer forces upper case, since a station name is a place name.

Station, FREQ and SQ open holding the last call in grey. Tapping into any of them clears that field at once — those three are replaced rather than edited, and deleting four digits by hand in turbulence is a waste of a hand. Typing turns the text dark.

Beside *Transmitted* sits **Copy from last call**, greyed and dashed, and it stays there. One tap puts all three values back in dark type and drops the cursor into the message, without any message of its own — the fields having filled is the confirmation. The station comes back with a trailing space and is then left alone: tapping into it keeps what was copied, so `ZURICH INFO` can be turned into `ZURICH INFO ARRIVAL` without retyping the first two words. FREQ and SQ still clear on a tap, because those two are replaced rather than extended. Empty, the two fields say what belongs in them — `FREQ XXX.XXX MHz` and `SQ XXXX` — which is a line of labels saved above them.

Beneath the station field are the station presets — *Info, Radar, Tower, Control, Approach* by default, and whatever you put in the setup instead. A tap inserts the word **where the cursor stands** rather than replacing the field, with exactly one space between it and what is already there, so `ZURICH` + *Info* + *Radar* becomes `ZURICH Info Radar` and never `ZURICH Info`.

A message preset is inserted with a trailing space, so the next word can simply be typed on.

The message presets sit under the message field and **differ by direction**: one list for calls received, another for calls transmitted, both in the setup under *ATC*. What you note down when a station calls you is rarely what you note down when you call it. Both start with the same list until you change them.

The standard phrases sit above the message field. Tapping one adds it, tapping it again takes it out, so QNH and Ops Normal are a toggle rather than something that accumulates.

The remark field on Ballast and Ressources is the same size as the ATC message field, since a remark is often the more important part of the entry.

Ressources puts current PIC and who is resting on the first line as panel switches, then battery, fuel cell and solar cell on the second, then methanol level, *Crew on O2* and the O2 level on the third. Only pilots the master actually named appear: leave the second name empty and the switches shrink to a single-handed flight.

Changing the PIC puts the other pilot on rest automatically, because that is what a handover normally means. *Nobody* is still one tap away for the stretches when both are awake.

A handover is asked about before it is filed. If the PIC named on this report differs from the one on the last, posting stops to ask *The pilot in command changes from A to B. Report it?*

On *Report the change* **two rows are written**. First a row of its own, of a kind called **PIC**, reading `PIC Change, new: B. Müller`. It is laid out like every other entry — the position, altitude, track and place sit in the footer of the row, not inside the message, so nothing is printed twice. It goes to the table only, never to WhatsApp — the crew learns of the change from the message that follows, and a chat does not need it twice. Then the resources report itself, as before, marked **Change of PIC** at the head of its line and of its WhatsApp message, and sent to whoever is ticked. Two rows are written, so two receipts appear above the post button in turn: *PIC change posted*, then *Crew status posted*.

The log therefore shows the moment of the handover as an event, rather than leaving it to be inferred from a name that quietly differs from the one before. **PIC** can be picked or left out of a printout like any other kind.

On *Just the reading* it is filed as an ordinary resources report with the new name, and no PIC row is written. A change of command is a fact of the flight, not a side effect of a tap, and the log should be able to show when it happened.

Battery and *Methanol level* both open the same list of percentages in a window, rather than a dropdown, so the two read alike and both are one tap away in gloves.

O2 Reserve is calculated rather than typed: volume x pressure, summed over the cylinders, which gives the litres of gas at one bar. **Each cylinder keeps its own full pressure**, taken from the setup table — a 10 litre bottle filled to 300 bar beside a 2 litre at 200 would otherwise be averaged, and the share of a full load would come out wrong. The button reads `1'700 lit (50%) ①`, thousands separated by an apostrophe — the reserve, its share of a full load, and the number of the bottle in use in a circle. It never shows more than 100 per cent, and it never shows pressures: those belong in the input window, which is where you report them. Tapping it opens one row per cylinder with its size, a field for the pressure, and a radio button marking which cylinder is **in use**.

The button only responds while **Crew on O2** is on. Reporting a pressure while nobody is breathing from the bottle records a number that means nothing, and the reserve figure is only of interest once it is being consumed — battery as a list in ten-percent steps with 75, 85 and 95 added, where the reading actually matters. The PIC and resting state stays active and is attached to every following entry of any type; battery, fuel cell and solar belong to the entry they were filed with.

Opening the Ballast tab always lands on **Drop** with **Sand** selected. Those are the overwhelming majority of entries, and a form that opens on Inventory because that is what happened an hour ago costs a tap every single time.

Other is a free note, up to four pictures, and a voice note. They are scaled down to 1024 pixels and about 60 per cent JPEG quality in the browser before anything is stored, because a queued entry has to survive in the device's local storage until there is a link again. Each picture is written to `data/media/<entry-id>-<n>.jpg` in the repository and the row records how many were attached and where they landed.

Add voice msg records through the microphone — press once to start, once again to stop, two minutes at most. The recording appears with a player so it can be listened to before posting, and is written to `data/media/<entry-id>-voice.webm`, or `.m4a` on iOS, where Safari records in that format instead.

A picture or a voice note travels with the message. `wa.me` carries text only, so when an Other entry has an attachment the app hands message and file together to the system share sheet, where WhatsApp is one of the destinations and the chat is picked there. Without an attachment the direct `wa.me` route is used as before, which is one tap fewer.

Attachments are not queued. Unlike the entries themselves, pictures and voice notes live only in memory until they are uploaded; posting without a connection files the note and says so, but the attachment is gone on the next reload. Record and photograph when there is a link, or expect to repeat it.

Each of the four forms opens with the last entry of its own kind, in a panel headed `Last Message | A. Wicki | 21:04:37Z` — what it was, who filed it and when. ATC spells the call out as `RCVD ZURICH INFO FREQ 124.700 MHz SQ 2450` with the wording beneath; Ballast shows the drop or the inventory; Ressources the crew and power state; Other the note. Where nothing of that kind has been reported yet, the panel simply reads `-`.

The reporter is named because the entry may well have come from the other pilot's device: the panel is read from the table, not from what this device happens to remember. A follow-up call never has to be reconstructed from memory, and never depends on who made the last one.

8. Position

The position on every row is the fix of the device that made the entry, taken at the moment of posting. Rows merged in from another iPad keep that iPad's position — nothing is ever re-stamped. Three columns carry it in short notation: `pos_lat` as `484532N`, `pos_lon` as `0072345E`, and `alt_ft` as WGS84 altitude in feet. The decimal degrees and metres stay alongside for map links and for anything that needs the raw value.

Track and ground speed come with every entry. iOS and Android supply both with the fix once the device is moving; when they do not, the app derives them from the previous fix, requiring at least three seconds and five metres between the two so that a stationary basket does not produce a random course. Both fields stay empty rather than guessing when neither source is available. The header carries the live values under the flight number.

9. Automatic position reports

Set an interval under *Ressources* in the setup and the app writes a **POS** row on its own at that rhythm — position, place, altitude and rate of climb — so the track of the flight survives the hours when nobody has a hand free. `0` turns it off, which is the default. POS rows go into the table and the printout like any other kind and can be filtered out of a printout on their own.

10. The Ops Normal clock

Send a transmitted call whose message contains *Ops Normal* and the app asks whether to keep the time. Pressing *Remind me* turns the button green and holds it for a second reading *Confirmed*, the same signal the post button gives, before the window closes: *Remind of the next Ops Normal in how many minutes?*, twenty by default and changeable in the window. When it falls due there is a tone and a message. It is the one call that has to happen on a clock, and the one most easily forgotten in weather.

11. Place names

On by default, and a setting of its own under *Ressources*. Switched on, each entry's position is turned once into something a reader can place — `near Szeged/HU` — which then appears under the position in the WhatsApp message, in the log, in the printout and in a `place` column.

It appears as `{place}` — `near Szeged/HU` — and as `{location}` for the bare `Szeged/HU`, in the heading of the Last Message panel after the time, in the log, in the printout and in a `place` column. A message on its way to WhatsApp waits up to two and a half seconds for the lookup rather than going out without it; beyond that it leaves anyway, because a message that arrives is worth more than a place name.

Lookups are queued one at a time with a second between them, as the service asks. A failure is not remembered — the next entry in the same square kilometre tries again — while an answer is, so a long flight costs a few dozen requests rather than one per entry.

Two things to weigh before switching it off. The lookup goes to OpenStreetMap's public service, so the coordinates of the entry leave the device; and it only works with a link. There is no offline place database that would fit in a web app. Failures are silent by design: the entry is written immediately and the place is filled in afterwards if an answer arrives, so a lost lookup costs nothing but an empty field.

Answers are cached on a coarse grid — about a kilometre — so a long flight costs a few dozen requests rather than one per entry, which keeps well inside what the service asks of its users.

12. When the satellites are gone

Jamming is not rare over parts of Europe, and a balloon that cannot say where it is becomes a problem for everyone. Two things follow from that.

Every kind of report goes out without a fix. Ballast, ATC, resources, notes and the automatic position rows are all posted with the position fields simply left empty rather than refused; a POS row written with no fix says so — *Position · no GPS signal*. Nothing in the app waits for a satellite.

And a position can be given by eye. *Report Position* sits under the picture and voice buttons on the Other screen. With a fix it asks first: report the satellite position, or place one by hand? The satellite answer writes an ordinary POS row and says *GPS position reported* — it does not disturb the rhythm of the automatic reports, it is simply one more row. With no fix the question is skipped and the map opens directly.

The map

The app shrinks to a small draggable button carrying the tally mark, and beneath it a map fills the screen — street or satellite, with zoom buttons. It opens on the last reported position.

- Every reported position is a dot with its time beside it: **blue from a satellite, red from the eye**. Tapping one shows its altitude, track and rate of climb for ten seconds.
- The dots are joined by the track flown, and from the last one the course is carried an hour forward, taken from the line between the **last two reported positions** — whatever kind they were — with cross ticks at 15, 30, 45 and 60 minutes.
- The last twenty positions are shown, adjustable in the setup. **An unbroken run of manual positions is always shown in full**, however long: that run is the whole picture when the satellites are gone.
- The map pinches to zoom as well as taking the plus and minus keys. **Whatever lies between the fingers stays between the fingers** — during the gesture the tiles are stretched about that point, and the map redraws at the nearest whole zoom when the fingers lift, holding the same ground under the same spot on the glass.
- Tapping the map places a crosshair, which can then be dragged until it sits over the place the crew believes it is.
- Tapping the floating button opens the report: altitude and rate of climb, prefilled from the last known values because both can be read off the barograph; track and speed, which are optional; and a slider for the estimated position uncertainty, in five steps. The circle for the step chosen is drawn around the crosshair straight away and **blinks slowly**, following the slider — so the choice is a picture of ground rather than a number. Climb and sink are two small keys beside the figure rather than a minus sign to be remembered with cold hands, and a **compass arrow beside the track turns with what is typed**, which shows a wrong digit before it is sent. Every prefilled field is selected when tapped, so a new figure simply replaces the old one.
- The rate of climb is typed in tenths: `20` becomes `2.0`, so the decimal point costs no keystroke.
- Before sending, the app asks once if any of altitude, track, speed or rate of climb is **empty**, and also if all four are still exactly the figures **carried over from the last report** — an unchanged altitude an hour later is usually an oversight rather than a reading. Three ways on: *Complete values* returns to the form, *Send as is* sends what stands there without a second press, and *Send only coordinates* clears the four and sends the position alone. - Track, projection, ticks and circles are all drawn thin with a white glow, the same treatment as the crosshair, so they stay legible on a pale street map and on a dark satellite image alike without shouting over the ground underneath.
- The panel sits at the top of the screen, narrow enough to leave the map beside and below it, and drops to the bottom when the crosshair is in the upper half. It has no title bar; *Report to CC* and *Close* share the bottom line.

The result is a **MANPOS** row, marked `(manual)` wherever its position appears. It has a WhatsApp template of its own in the setup, goes to the table and to whichever recipients are ticked on the Other screen, and appears in that screen's Last Message panel. It can be picked or left out of a printout like any other kind.

Afterwards the estimate is carried forward. With no fix, the next ATC or ballast report takes its position from the last manual one and marks it `(estimated at 2032Z)`, so nobody reads a terrestrial guess as a satellite one.

Tiles

Three views, chosen at the top right of the map and remembered between visits:

View	Source
Streets	OpenStreetMap standard tiles
Terrain	OpenTopoMap — contours and relief, the most useful of the three for reading ground from the air
Satellite	Esri World Imagery

None of them needs a key of yours to be typed in: the app ships with an ArcGIS access token, so the satellite layer goes through the service Esri asks applications to use from the first start. It can be replaced in the setup at any time: Two million tiles a month cost nothing and pay-as-you-go stays off unless switched on, so there is nothing to run up; a flight uses a few hundred. Emptying the field falls back to Esri's keyless address, which works but is not guaranteed.

A token in a public repository is readable by anyone. That is the price of a single-file app with no server, and the remedy is on Esri's side rather than in the code: an API key can be tied to a referrer, so a copy of the key only works from your own address. It is worth setting in the ArcGIS console — under the key's settings, *Referrers* — and it costs nothing. All three templates can be pointed elsewhere in *Tile sources* — a national orthophoto service such as swisstopo, basemap.at or IGN is sharper than any global layer inside its own borders, and needs no key either.

Tiles are kept two ways. Everything the map draws is held by the service worker on its way past. And a ring of tiles around the last reported position — **20 km by default, up to zoom 13**, both adjustable — is gathered quietly while there is a connection, **for the layer in use only**, capped at 400 tiles a run and spaced about eight a second. Switching the view starts a ring for the new layer. Nothing is ever fetched twice: what is in the cache is skipped.

Between the two there is no third mechanism to operate. Panning over a stretch of ground puts it in the cache because the map drew it; the ring keeps the surroundings of wherever the flight currently is. There is nothing to remember to press.

One thing to know about these sources. OpenStreetMap and OpenTopoMap are run on donated hardware, and both ask that their tiles not be fetched in advance for offline use — the OSM policy says so in as many words, and OpenTopoMap has far less capacity than OSM. Private, non-commercial use does not exempt anyone from that; the policies are about server load, not about money. What it does change is the scale: one crew, a few flights a year, a few hundred tiles a flight, is the load of a person looking at a map for a few minutes. That is the reason for the small radius, the cap and the pacing. If a layer ever stops answering, that is what has happened, and a keyed provider in *Tile sources* is the answer.

13. WhatsApp

All four forms list the recipients at the foot — on Ballast below both the drop and the inventory panel, since an inventory is as worth sending as a drop, under *Also send to WhatsApp* — the entry always goes to the table, and WhatsApp is the addition. **Ballast is off by default** even where the others are on: a drop happens many times an hour and would be noise in a chat, but the option is there as a fallback when the table cannot be reached, one chip per person, tapped on and off. Ballast entries never go to WhatsApp.

On ATC the coordinator sits on a line of its own above the rest of the list, so the one recipient that matters is never lost among the others.

Recipients sit in three columns in small type, each its own tappable field with a tick in front of it — faint when off, solid when on. Six recipients take two rows rather than six, and nothing can slip out of view; a long name is cut with an ellipsis rather than pushing the grid apart. The group keeps a full-width row of its own at the end. The label counts what will actually be sent: *Also send to WhatsApp · 3 recipients*.

In the setup, under the recipient rows, a line counts as you type: *3 with a usable number*, and names it when a row is being ignored because the number is too short. A recipient with no usable number cannot be messaged and is silently dropped, which is exactly the sort of thing that should not be silent. Pills are matched by number rather than by position — an earlier version matched by index and silently dropped the coordinator, which is the sort of thing that is only noticed when a message does not arrive.

The **ATC Coordinator** is listed first and only on ATC, and is the only one ticked there by default — an ATC call belongs with the coordinator, not with the whole crew. On Ressources and Other the coordinator does not appear and everybody else is ticked by default. Each choice is remembered per message type, so the pattern you settle into is the one you keep.

Recipients are defined in the setup, up to eight, each with a name and a number in international form with digits only, for example `41791234567`.

Above them sits a ninth, separate entry: the **ATC Coordinator**. It is offered on ATC entries only, as its own checkbox above the general one, and it is ticked by default there — an ATC call normally goes to the coordinator whether or not the rest of the list is in play. The two checkboxes are independent, so an ATC entry can go to the coordinator alone, to the list alone, to both, or to nobody. Ressources and Other never see the coordinator.

With the box ticked, posting first writes the entry as usual, then opens a sheet with the finished message and one button per recipient. An ATC entry reads:

```
*ATC | HB-QWV*
Station: ZURICH INFO
FREQ: 124.700 · SQ: 7000
Msg: QNH 1013, Ops Normal
21:04Z · 484532N 0072345E · 2340 ft · TC 293 · 12.3 kn
https://maps.google.com/?q=48.75890,7.39580
```

The first line is bold in WhatsApp, and the last is the position, which WhatsApp turns into a tappable link with a map preview card of its own.

The small confirmation that follows a post now sits at the right edge above the button instead of on top of it, and the mark in the footer holds still: it says where the queue stands, not whether a poll happens to be running this second. Green double check for sent, straw single check with the count for waiting, red cross when an attempt was refused.

The post button carries the whole exchange. The moment it is pressed it greys out, locks and reads *… wait for confirmation*, so a second tap cannot post the same entry twice. The entry appears in *Last Message* as soon as it is written. Then the button becomes the answer for three seconds — pale green *Reported to CC* when the row reached GitHub, pale yellow *Queued for later delivery (no internet connection currently)* when it is waiting, pale red *Failed* with the reason when something else went wrong — an expired token, a repository that is not there. The entry itself is never lost either way; the band only says where it stands.

Sending is immediate. With a box ticked, pressing *Report to CC* writes the entry and opens WhatsApp with the message already in the chat — no preview, no list, no extra tap. The tick survives from one message to the next, so a run of ATC calls to the coordinator is one button each. Only the send button inside WhatsApp is still yours to press; no browser can press that one. With more than one recipient the first opens straight away and a short list of the rest appears behind it, since WhatsApp takes one chat at a time.

The layout is yours

The setup holds one template per message type under *WhatsApp message layout*. Placeholders in braces are substituted, and lines behave in three ways:

- `{key}` — a line whose placeholders are all empty is dropped, so a field nobody reported costs nothing.
- `{!key}` — the line is kept whatever happens, and an empty value prints as `-`. The Msg line uses this, so every message shows what was said even when nothing was.

– A line made of nothing but values and separators, like `{time} · {pos} · {alt} · {tc} · {speed}`, keeps its own separator and simply loses the parts that are missing. Without a fix it collapses to the time alone, with no stray dots left behind.

Reset to default puts the five templates back.

The built-in templates are the standard, and your edits survive. A template you have written is kept across updates — losing someone's wording to a version bump would be worse than a missing placeholder. What the app does do is fill in a kind of message that has no saved template at all, which is how the Ballast template appeared when it was added without anybody having to reset anything. Placeholders added since your edit — the rate of climb, the place name, the direction of an ATC call — have to be put in by hand, or *Reset to default* brings the current wording back for all four.

Placeholder	Value
`{type}` `{callsign}` `{flight}` `{reporter}`	who and what
`{dir}` `{station}` `{freq}` `{squawk}` `{msg}`	the ATC fields
`{pic}` `{resting}` `{battery}` `{fuelcell}` `{solar}`	the Ressources fields
`{note}`	the free remark
`{time}`	UTC as `21:04Z`
`{pos}`	degrees, minutes and seconds, `484532N 0072345E`
`{posdm}`	degrees and minutes only, `4745N 00732E`
`{alt}`	GPS altitude in feet, rounded to ten
`{tc}` `{speed}`	track as `TC 235` and ground speed as `12.4 kn`
`{maps}`	link to the position on Google Maps

Without a position fix `{pos}`, `{alt}` and `{maps}` are empty: the map line disappears and the time line keeps just the time.

The row records `whatsapp = yes` and `whatsapp_to` with the names, so the table shows which entries were meant to go out. Whether they were actually sent happens inside WhatsApp and cannot be recorded here.

14. Master and followers

The master owns the setup and the log; the followers own their reporting. That is the whole division. Entries can only be changed or deleted on the master — a follower that taps a row is told so and asked to post a correction instead, which the master can then tidy. The log is the flight record, and one device has to be answerable for it. Only the master can unlock the protected part of the setup — a follower that tries is told so and pointed at the personal settings above, which are open to everyone. The master may be a shared or a personal device; that choice says who reports, not who owns the flight.

A follower posts everything a master posts, with no restriction, and every device sees it within five seconds. The one exception is the first entry of a new flight, which belongs to the master — that entry is what tells the crew the flight is really running.

The role can be handed over. Beside every other device in the list is *Make master*. The master offers, and **nothing changes until that device accepts**: the offer travels in the published setup, the other device asks its holder *Accept / Not now*, and only on a yes does it take the role and publish itself as the owner. The old master sees that in the file and steps down to follower on its own.

Two masters at once would overwrite each other's setup, so the change is deliberately one-way and confirmed at both ends. A declined offer is remembered and not asked again; the master can withdraw it by offering the role to somebody else, or by leaving it and carrying on.

A six-character code carries the whole flight. *Load current flight* is in the menu on every device and at the very top of the setup — on its own and across the full width where there is nothing else there yet, since a device with no flight has nothing else to press. *Synchronize data* only joins it once there is a table to synchronise — on a device just unpacked it would be a button with nothing behind it. Both are drawn the same way; neither is the loud one. On the master, above the device list, is *Join code*, and the same panel opens from the top right of the *Share setup* window. The master presses *Create*, reads six characters to the other device — `WUP9Q5`, in an alphabet with no O against 0 and no I against 1 — and that device has the flight: setup, token, presets, templates, and the log, which it reads from GitHub as soon as it has the token.

The setup travels **encrypted with the code itself**, so the relay that carries it holds a box it cannot open. It hands the box over **once** and deletes it, the code dies after a day, and *Revoke* kills it sooner. The panel counts the remaining life down by the minute rather than stating it once, and a tap on the copy key puts the code on the clipboard.

A code that has been collected stays visible, greyed and struck through, with the time it was taken: *Collected at 11:58Z. A code serves one device.* The master asks the relay every half minute whether its own code is still there — a look that does not take it — so the moment the other device has it, the panel says so. Copy and *Revoke* disappear, and *Create* becomes *New code*. An expired code is struck in the same way. Knowing which code was handed out, and when it was taken, is worth more than a tidy empty field. Twenty wrong guesses from one address and that address is shut out for an hour, which is what makes six characters enough.

The relay is a small Cloudflare worker, free to run; `join-worker/worker.js` and the twenty minutes of setup are described in the setup guide. Without it the code buttons say so and the link and QR code work as before.

Anyone holding the code holds your GitHub token until it expires — the code is exactly as confidential as the token inside it.

Devices on this flight sits below the GitHub connection on the master. It is read from two places: the entries themselves — who wrote, from which device, when, how many rows — and a small card every device leaves in `data/_seen/`, at most once every half hour. Without that card a follower that has been set up but has not yet reported would be invisible, which is exactly the moment one wants to see it. The master reads the cards when the menu is opened. A device that has not reported for **36 hours** is shown as quiet; the master is never counted out, since it may simply be watching. *Remove* puts a device on a list published with the setup; the next time it checks in it clears its own token and says so. Its entries stay in the table — removing a device is not rewriting the log. *Allow again* undoes it.

The master is whoever knows the master password. There is no other rule and no default.

Two passwords, doing two different jobs:

Password	What it does	Who has it
1234	opens the settings screen	everybody in the crew
5678	makes a device the master	one person, normally the PIC

The first time the app starts on a fresh device it asks the question outright: *Is this the master device?* Entering **5678** makes it the master. Choosing *Join as follower* — or getting the password wrong three times — makes it a follower. A device that was set up through an invitation link is a follower without being asked, because the link came from a master.

Nothing is assumed. A device that has not answered the question is not yet either, and the question comes back on the next start until it is answered.

Later changes go the same way: in the settings the role can be switched to Follower freely, but switching to Master asks for **5678** again. And because two masters would overwrite each other's setup, each published setup carries the identity of the device that wrote it; a master that sees a different one in the file says so on screen.

One device owns the flight setup and is the **Master** — normally the PIC's iPad. The others are **Followers**.

Menu -> *Share setup with another device* produces a link carrying the flight, the pilots, the ballast figures, the WhatsApp recipients and the message templates. Opening it on the other device shows what is about to be applied and asks for a yes; on yes that device takes the whole setup and marks itself a Follower. A checkbox decides whether the GitHub token travels with the link — with it the other device can send at once, without it the token has to be typed there. Send a link containing a token the way you would send a password.

What the link deliberately leaves alone: the reporter name, the personal-or-shared device mode, the colour scheme and the confirmation tone. Those belong to the device.

Every time the master saves settings it publishes them to `data/_setup.json`, and followers apply the change within about thirty seconds with a note on screen. Rename a pilot or start a new flight on the master and the crew follows without anyone opening a setup screen. Exactly one device may be the master; two would overwrite each other's publication.

15. Several devices at once

Every row carries `reporter` (who entered it), `device` (which iPad) and `source` (`app` for this tool).

Sending is a merge, never an overwrite: the app reads the file on GitHub, merges it with what is held locally by `id`, recalculates the ballast column over the whole set, and writes the result back. If another device wrote in the meantime, GitHub rejects the write and the app retries with the fresh version, up to three times.

The table is played back every five seconds. A drop made on one iPad shows up on the other within a few seconds, and the ballast figure accounts for both. A device that joins mid-flight, or whose local copy was cleared, gets the whole flight back with *Reload* in the log screen.

Polling that often is affordable because the app sends a conditional request: when nothing has changed, GitHub answers 304 with no body, and a 304 does not count against the hourly limit of 5000 requests. Only an actual change costs a request. Polling pauses while the app is in the background.

Tally rows. Anything appended to the same JSON file appears in the app and in the ballast calculation, provided it carries at least `id`, `ts_utc` and `type`. Give Tally-sourced rows `reporter: "Tally"` and `source: "tally"` so it is visible where they came from. A ballast row may be relative (`ballast_delta_kg`, negative for a drop) or an inventory (`ballast_action: "count"` with `total_ballast_kg`, which resets the running total).

Deletions are shared through `data/<flight-id>.deleted.json`, holding the ids that were removed with the time and the person who removed them. Every device applies that list, so a deleted row does not reappear from another iPad's copy, and the main table stays clean.

16. What carries over

Every form arrives holding what was reported last time, greyed and dashed: the station, frequency and squawk of the last call, the kilograms of the last drop, the bags and litres of the last inventory, the last battery, methanol and oxygen readings. Selections — sand or water, PIC and resting, fuel cell, solar cell, crew on oxygen, the ready-ballast percentage — arrive at the same setting they were left in.

The defaults come from the table, not from the device. They are read from the last entry of that kind on the whole flight, whoever made it and on whichever iPad. Hand the reporting over mid-flight and the second device opens with exactly what the first one last reported, within the five seconds it takes the table to arrive. Only free text is never carried over — remarks, notes and the ATC message are typed each time.

Touching any greyed field in a form accepts all of them at once, and nothing is asked again at posting. Notes and messages are never carried over: repeating yesterday's wording verbatim is worse than typing it again.

The footer names the last entry, its reporter and its kind, plus the state of the upload — `last msg 21:04:37Z/AW/Ballast | upload OK` when everything has reached GitHub, `x 3` when three are still queued.

17. Sending, and what happens without a link

There is nothing to switch on. Pressing **Report to CC** writes the entry to the device and sends it straight away. If there is no connection the entry is held in a queue — the header shows how many are waiting — and the queue goes out on its own as soon as the link returns, either on the browser's online event or on the next five-second cycle, whichever comes first.

Order is never in doubt: the file is always written as the complete set of rows sorted by `ts_utc`, so a queued entry lands in its correct place in the sequence rather than at the end. An entry made at 21:04 and sent at 21:11 still sits between 21:03 and 21:05 in the table.

The first sheet of a printout carries the Gas Balloon Team Switzerland logo at its top right, taken from `logo.png` beside the app; without that file it falls back to the app's own mark. The following sheets carry the small mark, so the first page is recognisable as the first.

Print the log builds an A4 portrait printout in its own window: you pick which of the four kinds it should contain, so an ATC-only log for the authority is one tap away. Entries run chronologically and numbered, each with its time, kind, content, reporter, position and track, one under the other as a table.

Column widths come from a colgroup rather than from the first row, so a page that opens with a full-width date heading lays itself out exactly like every other page.

A gas balloon flight can run for days, so the printout is broken by date. Each day opens with its own heading, in bold against the left edge of the table — `Wednesday, 20 May 2026` — and the numbering runs straight on across the break, because the entries are one flight and not three. The heading beside the callsign carries the span: `Wed 20 May 2026` for a single day, `Wed 20 May 2026 - Fri 22 May 2026` when it went further. Dates follow the UTC clock, like every time in the log, so a launch at 23:40 local does not appear to belong to the wrong day. A day heading never stands alone at the foot of a page — it travels with the first entry under it. Every page carries the flight in its heading with the mark on the right, and a footer reading `printed 2026-08-13 09:45Z by A. Wicki · v260825-08` on the left and `Page 2 / 3` on the right.

The document is named `Log HB-QWV GB-2026-01 printed 260813 0616Z (Ballast, ATC, Ressources, Other, POS)`, which is what a print-to-PDF ends up called: callsign, flight, the moment of printing and the kinds it contains, without opening it. After the last entry the table closes with `[no further log entries]`, so a printout can be seen to be complete.

Pagination is measured: the app takes the real height of a sheet, subtracts the heading, the footer and the margins, and fills each page with as many rows as actually fit. Where a browser gives no measurements it falls back to a fixed count rather than putting one entry per page.

Synchronize data at the top of the setup forces an attempt, and *Reload log* pulls the table without writing. Neither is needed in normal use. *Export the log* asks for the format first — CSV or JSON — and then for the destination: *Download* puts the file in the browser's downloads, *Share…* hands it to the system share sheet, which on an iPad is the way into Files, Mail or a chat. Share only appears where the browser supports handing over files.

18. Working without a link, and GitHub's limits

The app is complete offline. Its own files are cached, so it opens with no connection at all; entries are written to the device first and only then offered to GitHub. Without a link an entry is filed, marked with a straw check in the log and in the footer, and the post button says *Queued for later delivery*. Reporting continues unaffected — ballast, ATC, crew, notes, the automatic position reports, everything. When the connection returns, the queue goes out on its own,

in order, and the checks turn green without anyone pressing anything.

GitHub's numbers, and what the app does about them. A personal token is allowed 5,000 requests an hour, and a conditional read that answers `304 Not Modified` costs nothing at all — so the five-second poll is effectively free. The real constraint is the secondary limit on writing: **80 write requests a minute and 500 an hour**, counted across everything the account does.

Four things keep the app inside that:

– One push carries every waiting entry, so three drops in a minute cost one write, not three.

– Writes are spaced at least a second apart, as GitHub asks, and background pushes wait ten seconds between bursts — stretching to eighteen and then thirty as the hour's budget is spent.

– The CSV is written at most once a minute rather than on every entry, which halves the count. The JSON, which Notion reads, is always current.

– Writes are counted over a rolling hour and stopped at 440. Entries then stay queued and go out as the hour rolls on. If GitHub asks for a pause anyway, the `retry-after` it sends is obeyed.

The setup shows where you stand: *GitHub budget: 12 of 440 writes used this hour, 4,900 requests left on the token.*

Upgrading the GitHub account does not help. Free and Pro have exactly the same 5,000 requests an hour for a personal token; the higher 15,000 applies only to a GitHub App owned by an Enterprise Cloud organisation, which is a different kind of integration altogether. There is nothing to buy here, and with the measures above nothing to buy it for: a flight making an entry a minute for twenty hours uses well under half the hourly write budget.

19. The log

Log in the menu opens it: the whole flight, newest first, with time, content, reporter and a dot showing whether the row has been sent. Nothing else — transfer and export moved into the menu.

Each row names its kind in a small tag before the text — `BALLAST`, `ATC`, `POS` and so on — and a row of chips above the list filters by kind, each with its count: *All 128 · Ballast 61 · ATC 42 · Ressources 18 · POS 5 · PIC 2*. Only kinds that actually occur are offered. Reading the ATC calls of a long flight is then a tap rather than a search, and the choice is remembered between visits.

A row that also went out through WhatsApp says so after the reporter's name — *sent to WhatsApp*, previously the bare abbreviation `WA`.

Each row carries its transmission state at the right edge: a green double check when it has reached GitHub, a straw single check while it is waiting for a connection, and a red cross when an attempt was refused — an expired token, a repository that is not there. A cross is worth acting on; a single check only means the link is not up yet.

The ticks follow the queue on their own. When a batch finally goes out, every straw check in the list turns green within the second, without leaving the log or pulling to refresh. The list is only redrawn when the count of waiting or failed rows actually changes, so it does not fight your scrolling.

Tapping an entry opens it for editing. The editable fields depend on the type — kilograms and remark for a drop, bags, water and ready ballast for an inventory, station through message for ATC. Saving stamps `edited_at` and `edited_by`. The ballast column is recalculated over the whole flight after every edit and every deletion, so the figures stay consistent wherever in the sequence the change was made.

Deleting asks twice, states what is being removed and who recorded it, and warns that the deletion is shared with the other devices.

20. What the app writes

```
data/<flight-id>.json          the table
data/<flight-id>.csv           the same rows as CSV
data/<flight-id>.deleted.json  ids that were removed
```

Field	Content
`flight_id`, `callsign`	from the setup
`seq`	running number on the capturing device
`device`	device tag
`reporter`	who made the entry
`source`	`app`, or whatever an external writer sets, e.g. `tally`
`ts_utc`, `ts_local`	the same instant in UTC and with local offset
`type`	`Ballast`, `ATC`, `Ressources`, `Other` — the values used on the Tally form
`pos_lat`, `pos_lon`, `alt_ft`	position of the reporting device in short notation, `484532N` / `0072345E`, and altitude in feet
`tc_deg`, `speed_kt`	track over ground in degrees and ground speed in knots, one decimal
`lat`, `lon`, `alt_gps_m`	the same fix in decimal degrees and metres, for map links
`gps_acc_m`, `gps_age_s`, `gps_fix`	quality of the position at the moment of the entry
`device_mode`	`personal` or `shared`
`ballast_action`	`drop` or `count`
`ballast_medium`	`sand` or `water` on a drop
`ballast_delta_kg`	kilograms thrown, negative
`ballast_abs_kg`	on board after this entry, recalculated across all devices
`sand_kg`, `water_kg`, `total_ballast_kg`	inventory result
`sand_left_kg`, `water_left_kg`	what is left of each after this entry
`tally_diff_kg`	counted total minus what was expected at that moment
`inv_bags`, `inv_ready_pct`	what the inventory was built from
`atc_dir`	`RX` received, `TX` sent
`atc_station`, `atc_freq`, `atc_squawk`, `atc_msg`	message content
`pos_source`	`gps`, `manual` or `estimated`
`pos_est_at`	when the estimate it was carried from was made
`pos_unc_km`	how sure the crew was of a manual position, in kilometres
`crew_change`	`yes` when the entry was filed as a handover of command
`crew_pic`, `crew_rest`	crew state at the moment of the entry
`vs_ms`	rate of climb in metres per second
`res_battery_pct`, `res_fuel_cell`, `res_solar`	battery, fuel cell and solar cell as reported on a Ressources entry

Field	Content
`res_methanol_pct`	methanol level in per cent
`res_o2_crew`	whether the crew is on oxygen
`res_o2`	reserve as `560 l (70%)`
`res_o2_liters`, `res_o2_pct`	the same two figures on their own
`res_o2_detail`, `res_o2_bars`, `res_o2_inuse`	per cylinder, and which one is in use
`note`	free remark
`whatsapp`, `whatsapp_to`	set when the entry was marked for WhatsApp, and to whom
`attachments`, `attachment_paths`	how many pictures an Other entry carries, and where they were written
`edited_at`, `edited_by`	set when a row was changed after the fact
`id`	UUID, the stable key for Notion

`type` deliberately uses the spelling from the Tally form, including `Ressources`, so the existing Notion select options match without editing. To change it, edit the four `data-t` attributes in `index.html` and the matching strings in the post handler.

21. Polling from Notion

GitHub API — immediate, recommended

```
GET https://api.github.com/repos/<owner>/<repo>/contents/data/<flight-id>.json
Authorization: Bearer <token>
Accept: application/vnd.github.raw
```

Poll `<flight-id>.deleted.json` the same way and archive the matching rows in Notion. Use `id` as the unique key and upsert; the file always holds the complete set of rows, so appending produces duplicates.

raw.githubusercontent.com — public repositories only, delayed

```
GET https://raw.githubusercontent.com/<owner>/<repo>/main/data/<flight-id>.csv
```

Behind a CDN with roughly a five minute cache. Fine after the flight, too slow during it.

22. Beginning a new flight

Begin new flight appears **only on the master**, at the foot of the unlocked settings, and asks for the master password `5678`, then for the name of the new flight, then twice for confirmation — naming how many entries the old flight holds and how many of them are still queued.

Nothing is moved and nothing is overwritten. The old flight's files are named after it, so they simply stay where they are; they are recorded in `data/_flights.json` with the time they were closed, the number of rows and who closed them, which gives you an index of the season. Before closing, whatever is still queued is sent one last time. If there is no connection and rows are still waiting, the app says so and asks again before going on — those rows would be lost, because clearing the local table is what makes the new flight start empty.

The new flight ID is published to the followers along with the rest of the setup. A follower that sees the flight change files its own table away under `bsk.archive.<flight>` on the device and then clears it, so nobody carries yesterday's entries into today's flight under the new name.

The first entry of a flight belongs to the master. A follower trying to make it is told *New flight must be initiated by the master device*; the master is reminded once that the flight has been opened and asked to look over the settings, and the setup screen opens for that. From the second entry on, anyone reports.

23. Language

Device mode, night colour, confirmation tone and language sit above the password, because they are personal preferences rather than flight settings and every crew member may want their own. A device whose holder does not know the settings password is a **follower** on a **personal device** — that is what the app assumes when the master question is declined, and neither can be changed without the password.

The setup switches the interface between English and German, above the password, because it is a personal preference rather than a flight setting and every crew member may want a different one.

Every label, button, hint, dialog and message in the app follows the switch. **Only the interface changes:** everything written to GitHub and everything sent to WhatsApp stays English: the column names, the values in `type`, the message templates. A table that changed language depending on who happened to make the entry would be unusable, and the ATC coordinator should not have to guess which language the next message arrives in.

24. Keeping the crew on the current build

The service worker takes a new version over the moment it is installed, and the app now looks for one at every start, whenever it comes back to the foreground, and every quarter of an hour. When one has arrived it says so: *A newer version is ready — close the app and open it again.*

That message is the answer to a confusion worth knowing about. **What you see on screen may not be what is in the repository.** An installed web app keeps running the version it started with until it is closed and opened again — reloading a page inside it is not enough. If a change you asked for seems not to have happened, check the version in the bottom right corner first, and close the app from the app switcher before opening it again.

25. When a change does not appear

The service worker caches each file on its own and carries on past one that is missing. That sounds like housekeeping and is not: `cache.addAll()` rejects as a whole if a single request fails, and a worker whose installation fails is never activated — the app then keeps serving the **previous version indefinitely**, and every change made since is invisible. One icon missing from the server is enough. The install now notes what it could not fetch in the console and takes over anyway.

So when something you asked for is not there, the order to check is:

1. The version in the bottom right corner. If it is not the one you deployed, nothing else matters yet.
2. The browser console for `404` on any file of the app. A missing file means the upload was incomplete — the `icons` folder is the usual one, because a drag-and-drop upload does not always carry a folder with it.
3. Close the installed app from the app switcher and open it again. A reload inside it is not enough.

`GET .../data/_seen?ref=main 404` in the console is not a fault: it says no device has left its card in the repository yet. It disappears with the first one.

26. Limits worth knowing before takeoff

- **GPS in the background:** once the app is not in the foreground and the display locks, iPadOS suspends location updates. The app holds a wake lock while it is active. This records events, not a track — use a dedicated logger for a gapless trace.
- **Altitude:** `alt_gps_m` is GPS altitude above the WGS84 ellipsoid, not the altimeter reading. The altimeter governs what is reported to ATC.

– **Rate limit:** with two iPads and a Notion poll on the same token, the hourly budget is comfortable as long as the conditional requests keep returning 304. Frequent writes are what cost — each post is roughly four requests.
– **The GitHub token stays put.** Leaving the token field empty when saving means *leave it as it is*, not *clear it* — a fine-grained token cannot be retyped from memory, and losing it by accident would ground the device until a new one is made. To replace it, paste the new one over the old.

The passwords are 1234 and 5678 and both live in the source. They guard against a mistaken tap in the basket and against a second device declaring itself master, not against anyone who reads the file. Change `PASS` and `MASTER_PASS` in `index.html` for different ones — and if you do, tell the crew, because a device that cannot answer the master question ends up a follower. - **Token on the device:** stored unencrypted in the browser. If the iPad is lost, revoking the token is enough. To avoid it entirely, put a Cloudflare Worker in front holding the token server side and point the API host in the code at the Worker. - **The pill floats inside the app only** on phones and tablets — see section 4. Slide Over on iPad and Pop-up view on Samsung are the routes that put the whole window on top; document picture-in-picture on a laptop is the only true always-on-top.

27. Version

The build stamp sits in the bottom right of every screen, in the form `v<YYMMDD>-<nn>` — the date written backwards, then a counter that restarts at 01 each day and goes up with every build released that day. The date leads, so `v260813-01` is newer than `v260812-26` despite the smaller counter. `v260825-12` is the twelfth build of 25 August 2026.

It lives in two places that must be kept in step: the `APP_VERSION` constant near the top of the script in `index.html`, and the cache name `V` in `sw.js`. `readme.html`, `setup.html` and the PDFs are generated from `README.md` and `SETUP.md` and are regenerated on every build, so the printed manual is never behind the app. Bumping the cache name is what forces the service worker to fetch the new files rather than serve the old ones, so a build with an unchanged cache name may not reach a device that already has the app installed.

28. License

Two things the licence spells out that are easy to get wrong:

Configuration is not modification. Entering your flight, crew, ballast, WhatsApp and GitHub settings, switching language or colour scheme, editing the message templates in the setup, and dropping your own logo file next to the app are all ordinary use. You are meant to do all of that.

The flight data is yours. The JSON, CSV and media files the app writes into your repository are your records, not part of the licensed software.

Third-party components: none. No libraries, no fonts, no frameworks, no trackers, nothing from a CDN. The app talks to the GitHub API with your token, to the browser's geolocation service, and to the wa.me links you tap. That is the whole list, which is also why it works with no signal and why there is nothing to keep up to date but the app itself.

29. Files

File	What it is
`index.html`	the whole app — markup, style, logic, the QR encoder and the map

File	What it is
`sw.js`	the service worker: keeps the app available offline and holds the map tiles
`manifest.webmanifest`	name, colours and icons for the home screen
`icons/`, `favicon.ico`, `logo.png`	the tally mark in every size a device asks for, and the club logo for the printout
`readme.html`, `setup.html`, `notion.html`, `license.html`	this manual, the setup guide, the Notion guide and the licence, as pages
`README.md`, `SETUP.md`, `NOTION.md`, `LICENSE`	the sources those pages are made from
`Basket-Reporting-Manual.pdf`, `Setup-data-repository.pdf`, `Notion-integration.pdf`	the same three documents to print
`notion-sync/sync.mjs`, `notion-sync/notion-sync.yml`	the script and workflow that push the table into Notion — they belong in the **data** repository
`join-worker/worker.js`	the Cloudflare worker that carries a join code — it belongs in Cloudflare, not in either repository
`.nojekyll`	keeps GitHub Pages from rebuilding the folder

Everything except the first four is documentation or lives elsewhere: the app runs on `index.html`, `sw.js`, `manifest.webmanifest` and `icons/`.